```python
import csv, math, time, tracemalloc, random, platform, sys, urllib.request, os
from statistics import median

URL = "https://raw.githubusercontent.com/jpatokal/openflights/master/data/airports.dat"
PATH = "airports.dat"
try:
    urllib.request.urlretrieve(URL, PATH)
    points=[]
    with open(PATH, newline="", encoding="utf-8", errors="replace") as f:
        for row in csv.reader(f):
            try: points.append((float(row[7]), float(row[6])))
            except (ValueError, IndexError): continue
    print("OpenFlights dataset downloaded")
except Exception:
    random.seed(2026)
    points=[(random.uniform(-180,180), random.uniform(-90,90)) for _ in range(10000)]
    print("Download unavailable; deterministic fixture used")
print("Valid coordinate records:", len(points))
print("Python:", sys.version.split()[0])
print("Platform:", platform.platform())
```

```
OpenFlights dataset downloaded
Valid coordinate records: 7698
Python: 3.13.15
Platform: Linux-6.6.122+-x86_64-with-glibc2.35
```

```
DISTANCE_COUNT=0
def distance(p,q):
    global DISTANCE_COUNT; DISTANCE_COUNT+=1
    return math.hypot(p[0]-q[0], p[1]-q[1])

def brute_force(points):
    if len(points)<2: return None, float("inf")
    best_pair=None; best_d=float("inf")
    for i in range(len(points)-1):
        for j in range(i+1,len(points)):
            d=distance(points[i],points[j])
            if d<best_d: best_d=d; best_pair=(points[i],points[j])
    return best_pair,best_d

def dc_rec(px,py,threshold=3):
    n=len(px)
    if n<=threshold: return brute_force(px)
    mid=n//2; midx=px[mid][0]; left_x=px[:mid]; right_x=px[mid:]
    left_counts={}
    for pt in left_x: left_counts[pt]=left_counts.get(pt,0)+1
    left_y=[]; right_y=[]
    for pt in py:
        if left_counts.get(pt,0)>0: left_y.append(pt); left_counts[pt]-=1
        else: right_y.append(pt)
    pl,dl=dc_rec(left_x,left_y,threshold); pr,dr=dc_rec(right_x,right_y,threshold)
    if dl<=dr: best_pair,delta=pl,dl
    else: best_pair,delta=pr,dr
    strip=[pt for pt in py if abs(pt[0]-midx)<delta]
    for i in range(len(strip)):
        for j in range(i+1,min(i+8,len(strip))):
            if strip[j][1]-strip[i][1]>=delta: break
            d=distance(strip[i],strip[j])
            if d<delta: delta=d; best_pair=(strip[i],strip[j])
    return best_pair,delta

def divide_and_conquer(points):
    px=sorted(points,key=lambda p:(p[0],p[1])); py=sorted(points,key=lambda p:(p[1],p[0])); return dc_rec(px,py,3)

def hybrid(points,threshold):
    px=sorted(points,key=lambda p:(p[0],p[1])); py=sorted(points,key=lambda p:(p[1],p[0])); return dc_rec(px,py,threshold)

def benchmark(func,*args,repeats=3):
    global DISTANCE_COUNT; times=[];peaks=[];counts=[];out=None
    for _ in range(repeats):
        DISTANCE_COUNT=0;tracemalloc.start();t=time.perf_counter();pair,d=func(*args);elapsed=time.perf_counter()-t;_,peak=tracemalloc.get_traced_memory();tracemalloc.stop()
        times.append(elapsed*1000);peaks.append(peak/1024);counts.append(DISTANCE_COUNT);out=(pair,d)
    return {"median_time_ms":median(times),"median_peak_kib":median(peaks),"median_distance_count":median(counts),"pair":out[0],"distance":out[1]}
```

```
candidates=[2,4,8,16,24,32,48,64,96]
calibration=points[:min(5000,len(points))]
threshold_results={}
for T in candidates:
    threshold_results[T]=benchmark(hybrid,calibration,T,repeats=3)
best_t=min(threshold_results,key=lambda t:threshold_results[t]["median_time_ms"])
for T,r in threshold_results.items(): print(T, round(r["median_time_ms"],3), int(r["median_distance_count"]))
print("Selected T*:",best_t)
```

```
2 215.1 5536
4 150.701 6447
8 118.468 10663
16 116.471 22233
24 110.671 46424
32 116.798 46424
48 160.346 95172
64 164.218 95172
96 501.806 192821
Selected T*: 24
```

```python
sizes=[100,500,1000,2000,5000,10000]
all_results={}
for n in sizes:
    if n>len(points): break
    pts=points[:n]
    if n<=5000:
        bf_repeats=1 if n==5000 else 3
        all_results[(n,"Brute Force")]=benchmark(brute_force,pts,repeats=bf_repeats)
    all_results[(n,"Divide & Conquer")]=benchmark(divide_and_conquer,pts,repeats=3)
    all_results[(n,"Hybrid")]=benchmark(hybrid,pts,best_t,repeats=3)
    print("N=",n,all_results[(n,"Divide & Conquer")],all_results[(n,"Hybrid")])
    if n<=5000: print("Brute Force:",all_results[(n,"Brute Force")])
```

```
N= 100 {'median_time_ms': 2.9259319999823674, 'median_peak_kib': 9.53125, 'median_distance_count': 110, 'pair': ((-75.66919708251953, 45.3224983215332), (-75.5635986328, 45.521701812699995)), 'distance': 0.2254618891879679} {'median_time_ms': 1.4725670000075297, 'median_
Brute Force: {'median_time_ms': 7.191159000001335, 'median_peak_kib': 0.7890625, 'median_distance_count': 4950, 'pair': ((-75.5635986328, 45.521701812699995), (-75.66919708251953, 45.3224983215332)), 'distance': 0.2254618891879679}
N= 500 {'median_time_ms': 12.584629999992103, 'median_peak_kib': 42.01953125, 'median_distance_count': 532, 'pair': ((2.8141698837280273, 36.50360107421875), (2.87611, 36.545799)), 'distance': 0.07494826845248391} {'median_time_ms': 8.079945000019961, 'median_peak_kib': 4
Brute Force: {'median_time_ms': 235.338186999968, 'median_peak_kib': 0.27734375, 'median_distance_count': 124750, 'pair': ((2.8141698837280273, 36.50360107421875), (2.87611, 36.545799)), 'distance': 0.07494826845248391}
N= 1000 {'median_time_ms': 29.33778699997447, 'median_peak_kib': 83.84375, 'median_distance_count': 1081, 'pair': ((28.222499847399998, -25.829999923699997), (28.164680372299997, -25.8097000122)), 'distance': 0.06135499673020482 4} {'median_time_ms': 17.917750999970394, '
Brute Force: {'median_time_ms': 978.9739130000044, 'median_peak_kib': 0.30859375, 'median_distance_count': 499500, 'pair': ((28.164680372299997, -25.8097000122), (28.222499847399998, -25.829999923699997)), 'distance': 0.06135499673020482 4}
N= 2000 {'median_time_ms': 60.53249600000754, 'median_peak_kib': 168.47265625, 'median_distance_count': 2175, 'pair': ((29.258899688720703, -1.6771999597549438), (29.238500595092773, -1.6708099842071533)), 'distance': 0.021376501312010 87} {'median_time_ms': 66.77530499999
Brute Force: {'median_time_ms': 4693.766159999996, 'median_peak_kib': 0.30859375, 'median_distance_count': 1999000, 'pair': ((29.238500595092773, -1.6708099842071533), (29.258899688720703, -1.6771999597549438)), 'distance': 0.021376501312010 87}
N= 5000 {'median_time_ms': 318.5354830000051, 'median_peak_kib': 484.72265625, 'median_distance_count': 6122, 'pair': ((29.258899688720703, -1.6771999597549438), (29.238500595092773, -1.6708099842071533)), 'distance': 0.021376501312010 87} {'median_time_ms': 209.81601199
Brute Force: {'median_time_ms': 29139.150452000024, 'median_peak_kib': 4.4755859375, 'median_distance_count': 12497500, 'pair': ((29.238500595092773, -1.6708099842071533), (29.258899688720703, -1.6771999597549438)), 'distance': 0.021376501312010 87}
```

```python
for n in [100,500,1000]:
    pts=points[:n]
    _,db=brute_force(pts); _,dd=divide_and_conquer(pts); _,dh=hybrid(pts,best_t
    assert abs(db-dd)<=1e-12 and abs(db-dh)<=1e-12
    print(n,"Correctness: PASS")
```

```
100 Correctness: PASS
500 Correctness: PASS
```

```python
for n in [100,500,1000]:
    pts=points[:n]
    _,db=brute_force(pts); _,dd=divide_and_conquer(pts); _,dh=hybrid(pts,best_t
    assert abs(db-dd)<=1e-12 and abs(db-dh)<=1e-12
    print(n,"Correctness: PASS")
```

```
100 Correctness: PASS
500 Correctness: PASS
```